



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное бюджетное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана**
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 3
по дисциплине «Анализ алгоритмов»

Тема Поиск по словарю

Студент Пантелеев Василий Максимович

Группа ИУ7-52Б

Преподаватели Строганов Дмитрий Владимирович

Москва, 2024

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Линейный поиск	4
1.2 Бинарный поиск	4
2 Конструкторская часть	5
2.1 Описание алгоритмов	5
2.2 Анализ трудоемкости	8
2.2.1 Линейный поиск	8
2.2.2 Бинарный поиск	8
3 Технологическая часть	9
3.1 Средства реализации	9
3.2 Реализация алгоритмов	10
3.3 Функциональные тесты	11
4 Исследовательская часть	12
4.1 Технические характеристики ЭВМ	12
4.2 Сравнение алгоритмов	12
ЗАКЛЮЧЕНИЕ	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18

ВВЕДЕНИЕ

Цель лабораторной работы — сравнить два алгоритма поиска элемента (полного перебора и бинарного поиска) при работе со словарем. Для достижения поставленной цели необходимо выполнить следующие задачи:

- выполнить оценку трудоемкости разрабатываемых алгоритмов;
- реализовать алгоритмы нахождения значения в словаре (алгоритм полного перебора и бинарный поиск);
- выполнить сравнительный анализ сложности двух алгоритмов на основе замеров количества сравнений для каждого элемента массива и данных лучшего и худшего случаев.

1 Аналитическая часть

В данном разделе будут рассмотрены теоретические аспекты алгоритмов нахождения заданного значения в словаре.

1.1 Лине́йный поиск

Линейный поиск проходит через массив, начиная с первого элемента, и сравнивает каждый элемент с искомым. Если элемент найден, алгоритм возвращает его индекс. Если нет — продолжается поиск до конца массива. В худшем случае требуется проверить все элементы массива. Элементы, расположенные в начале массива будут найдены быстрее тех, что расположены в конце.

Пусть массив $A = \{a_1, a_2, \dots, a_n\}$, а искомый элемент x . Тогда алгоритм линейного поиска можно записать следующим образом:

- для каждого элемента $a_i \in A$, если $a_i = x$, то вернуть индекс i ;
- если x не найден после проверки всех элементов, вернуть -1 .

1.2 Бина́рный поиск

Бинарный поиск работает на отсортированных массивах. Алгоритм разделяет массив пополам и проверяет центральный элемент. Если центральный элемент совпадает с искомым, то поиск завершён. Если искомый элемент меньше центрального, алгоритм продолжает работу в левой половине массива, если больше — в правой. Этот процесс повторяется до тех пор, пока не будет найден элемент или интервал для поиска не станет пустым.

Вывод

В данном разделе были рассмотрены теоретические аспекты алгоритмов нахождения заданного значения в словаре.

2 Конструкторская часть

В данном разделе будут приведены схемы алгоритмов нахождения элемента в массиве

2.1 Описание алгоритмов

На вход алгоритмов подаются 2 строки - *str1* и *str2*, на выходе получается искомое расстояние. Далее приведены схемы рекурсивных и матричных алгоритмов нахождения расстояний Левенштейна и Дamerau-Левенштейна.

На вход алгоритмы получают массив *arr* и значение *elem*, которое необходимо найти. На выходе алгоритмы возвращают найденный индекс, а также количество сравнений. В случае отсутствия заданного элемента в массиве алгоритмы, в качестве найденного индекса, возвращают -1 .

На схемах 2.1 — 2.2 приведены описания алгоритмов нахождения значения в массиве: использующий полный перебор, использующий бинарный поиск.

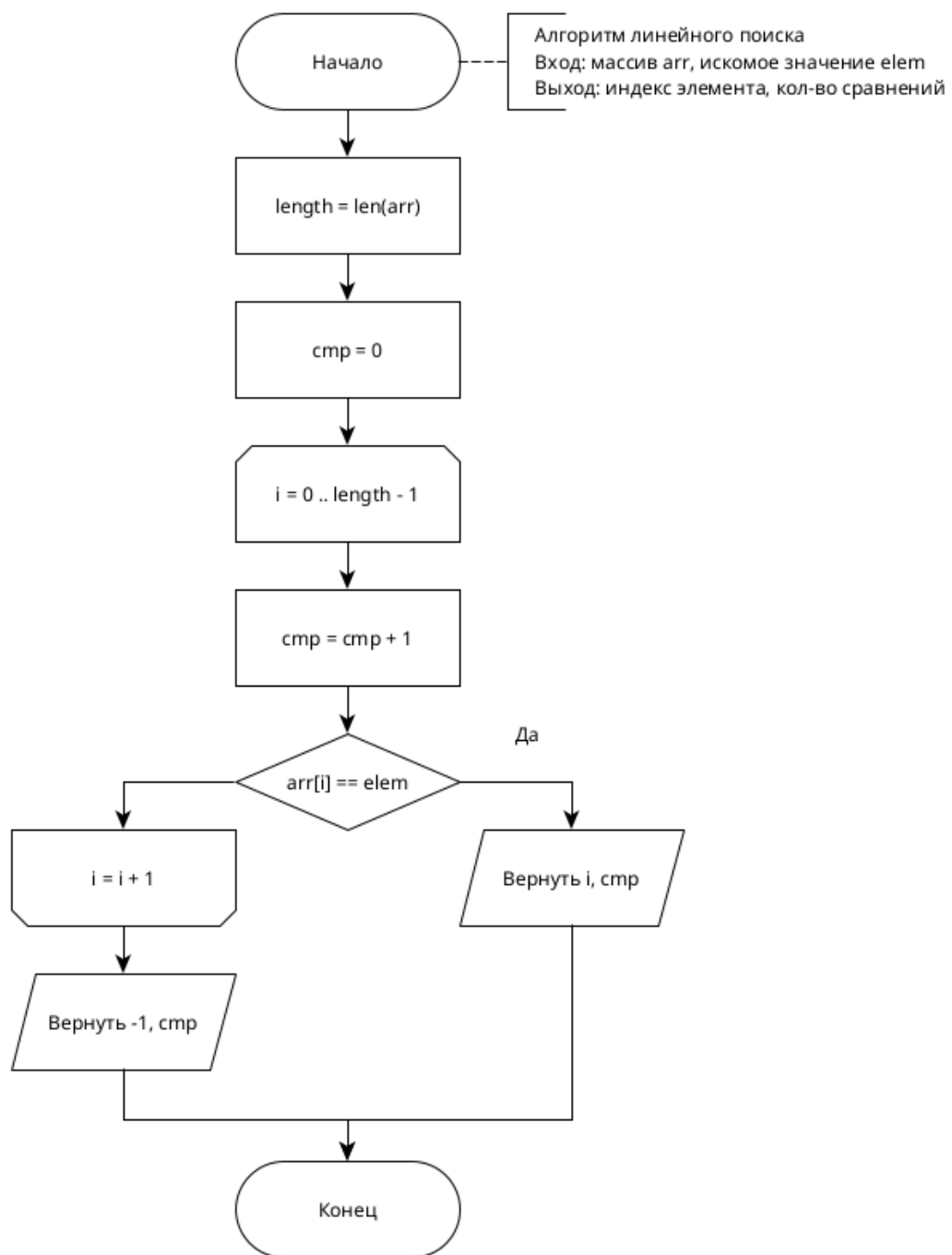


Рисунок 2.1 — Схема алгоритма поиска полным перебором

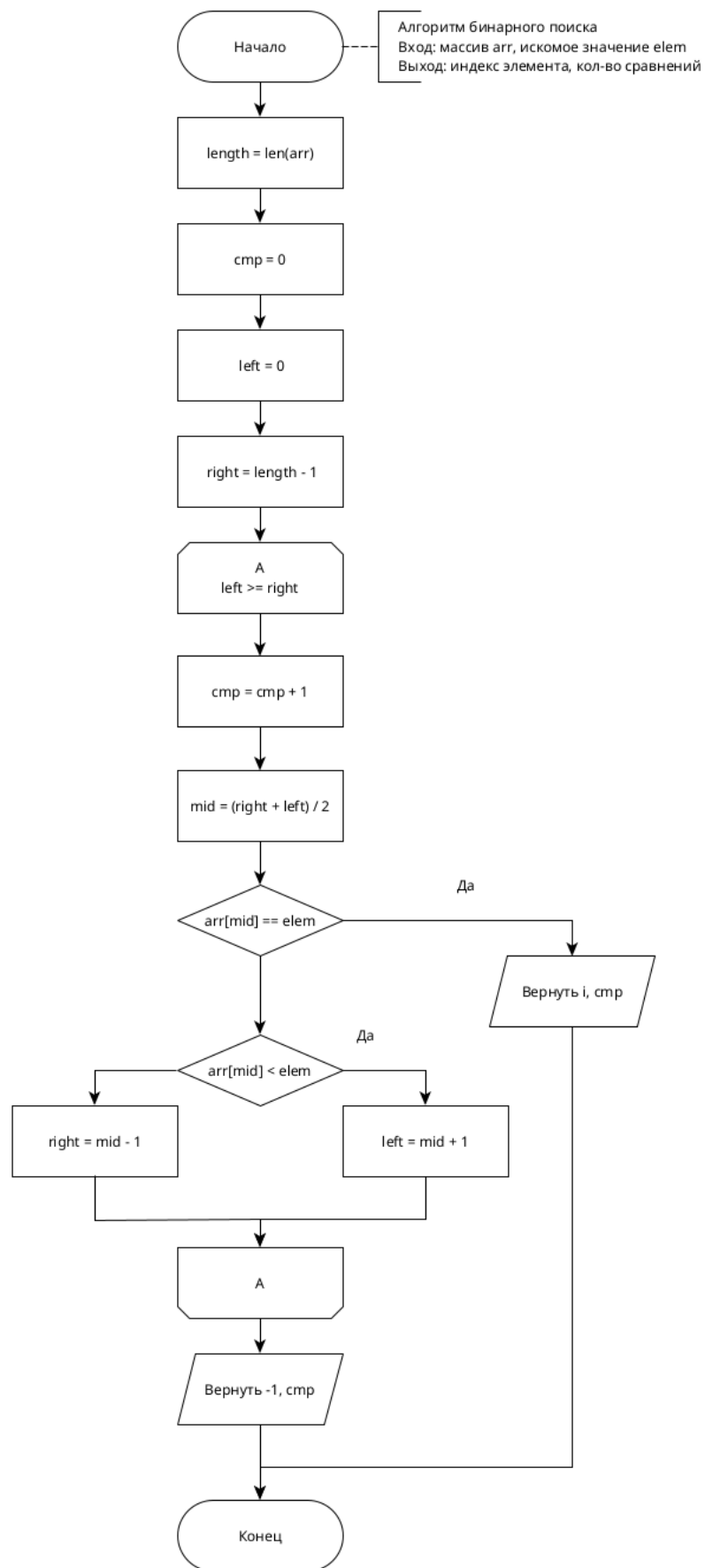


Рисунок 2.2 — Схема алгоритма поиска бинарным поиском

2.2 Анализ трудоемкости

2.2.1 Линейный поиск

Линейный поиск проверяет каждый элемент массива последовательно, пока не найдёт искомый элемент или не проверит все элементы массива.

1. В **лучшем случае**, элемент может быть найден на первом месте массива. В этом случае потребуется только одно сравнение, и временная сложность будет равна $O(1)$.
2. В **худшем случае**, элемент либо находится на последней позиции массива, либо отсутствует в массиве. Тогда алгоритм должен проверить все n элементов массива, что приводит к временной сложности $O(n)$.

2.2.2 Бинарный поиск

Бинарный поиск работает по принципу деления массива пополам на каждом шаге. В начале поиска массив имеет n элементов, и после каждой итерации количество элементов, которые нужно проверить, сокращается в два раза. Рассмотрим временную сложность в различных случаях:

1. В **лучшем случае**, искомый элемент находится точно в середине массива после первой итерации. Это требует лишь одного сравнения, и временная сложность равна $O(1)$.
2. В **худшем случае**, поиск продолжается до тех пор, пока размер массива не будет уменьшен до 1 элемента. Количество делений массива пополам можно выразить как $\log_2 n$, где n — размер массива. Таким образом, время работы в худшем случае равно $O(\log n)$.

Вывод

В данном разделе были представлены схемы алгоритмов нахождения элемента в словаре, а также оценены трудоемкости для данных алгоритмов в лучшем и худшем случаях.

3 Технологическая часть

В этом разделе обоснованы средства реализации, а так же представлены реализации алгоритмов и функциональные тесты.

3.1 Средства реализации

Для реализации алгоритмов в этой лабораторной работы был выбран язык *Python*^[1] по следующим причинам:

- python предоставляет необходимые структуры данных, такие как списки (массивы);
- поддерживаемые библиотеки, такие как *matplotlib* позволяют визуализировать результаты исследования.

3.2 Реализация алгоритмов

В листингах 3.1 — 3.2 представлены реализации алгоритмов.

Листинг 3.1 — Алгоритм использующий полный перебор

```
def search_linear(arr, elem):
    length = len(arr)

    if length == 0:
        return -1, 0

    cmp = 0

    for i in range(length):
        cmp += 1

        if arr[i] == elem:
            return i, cmp

    return -1, cmp
```

Листинг 3.2 — Алгоритм использующий бинарный поиск

```
def search_binary(arr, elem):
    length = len(arr)

    if length == 0:
        return -1, 0

    cmp = 0
    left = 0
    right = len(arr) - 1
    while left <= right:
        cmp += 1
        mid = (left + right) // 2

        if arr[mid] == elem:
            return mid, cmp
        elif arr[mid] < elem:
            left = mid + 1
        else:
            right = mid - 1

    return -1, cmp
```

3.3 Функциональные тесты

В таблице 3.1 приведены функциональные тесты, на которых тестировалась программа.

Таблица 3.1 — Функциональные тесты, на которых были протестированы алгоритмы (в результате – индекс элемента).

Массив	Искомый элемент	Линейный поиск	Бинарный поиск
[1, 2, 3, 4, 5]	1	0	0
[1, 2, 3, 4, 5]	5	4	4
[1, 2, 3, 4, 5]	3	2	2
[]	3	-1	-1
[1, 2, 3, 4, 5]	6	-1	-1

Вывод

В технологической части были представлены листинги реализованных алгоритмов, включая линейный и бинарный поиск по массиву. Также были проведены функциональные тесты, которые подтвердили корректность работы алгоритмов на различных входных данных, включая граничные случаи. Результаты тестов показали, что разработанные реализации соответствуют поставленным требованиям.

4 Исследовательская часть

В этом разделе приведен сравнительный анализ алгоритмов по количеству операций сравнения, необходимых для того, чтобы найти искомый элемент в массиве.

4.1 Технические характеристики ЭВМ

Все замеры проводились на ЭВМ, характеристики которой приведены ниже:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16;
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS;
- оперативная память: 16,0 ГБ.

4.2 Сравнение алгоритмов

Размер массива, на котором проводилось исследование равен 1023, согласно варианту. Предварительно массив был заполнен случайным образом с помощью модуля *random*^[2]. Гистограммы построены с помощью библиотеки *matplotlib*^[3]. Гистограммы 4.1—4.2 показывают количество сравнений для каждого элемента словаря в линейном и бинарном поиске. На рисунке 4.3 приведена отсортированная гистограмма из рисунка 4.2.

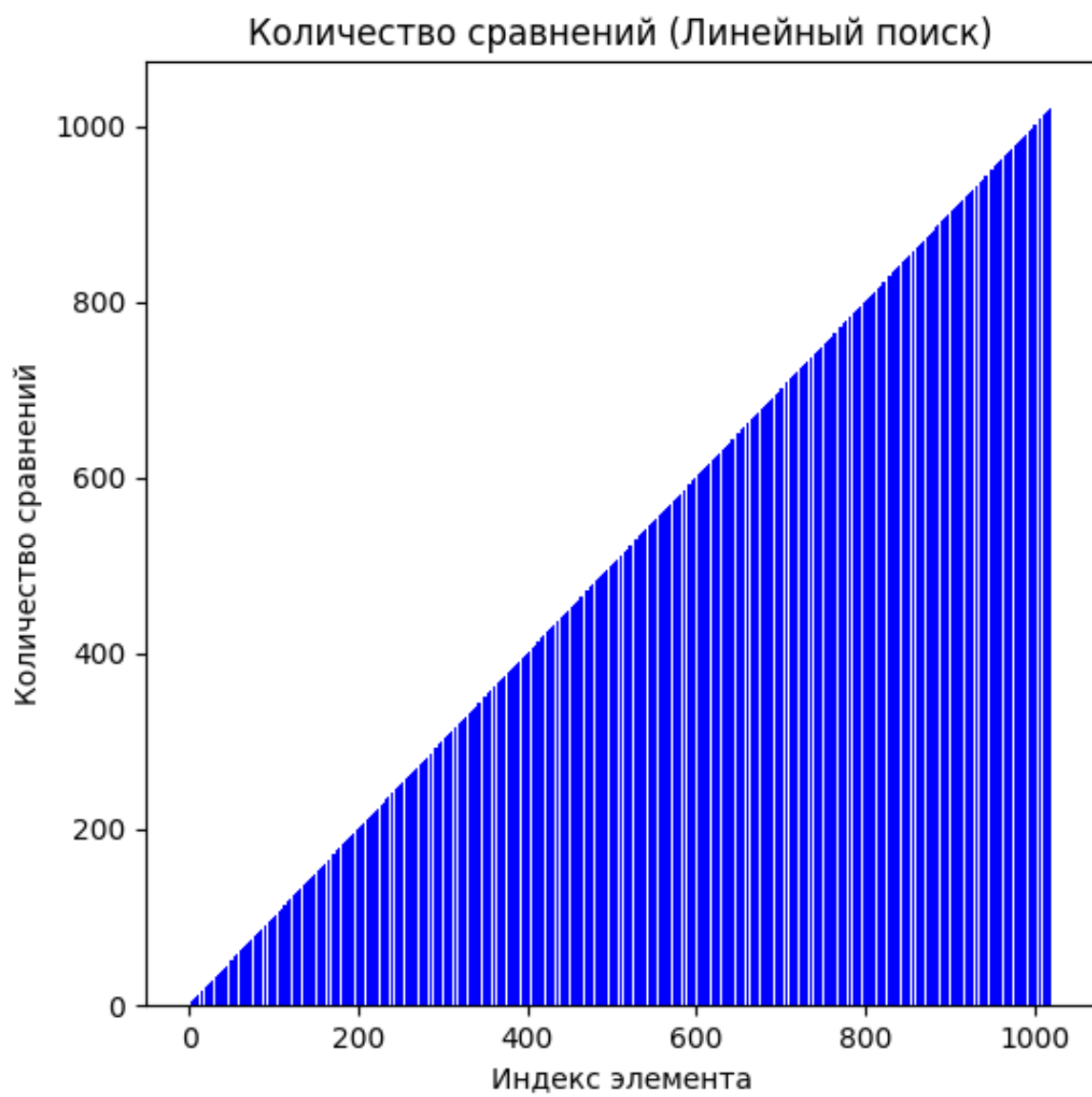


Рисунок 4.1 — Сравнения для каждого элемента в алгоритме полного перебора

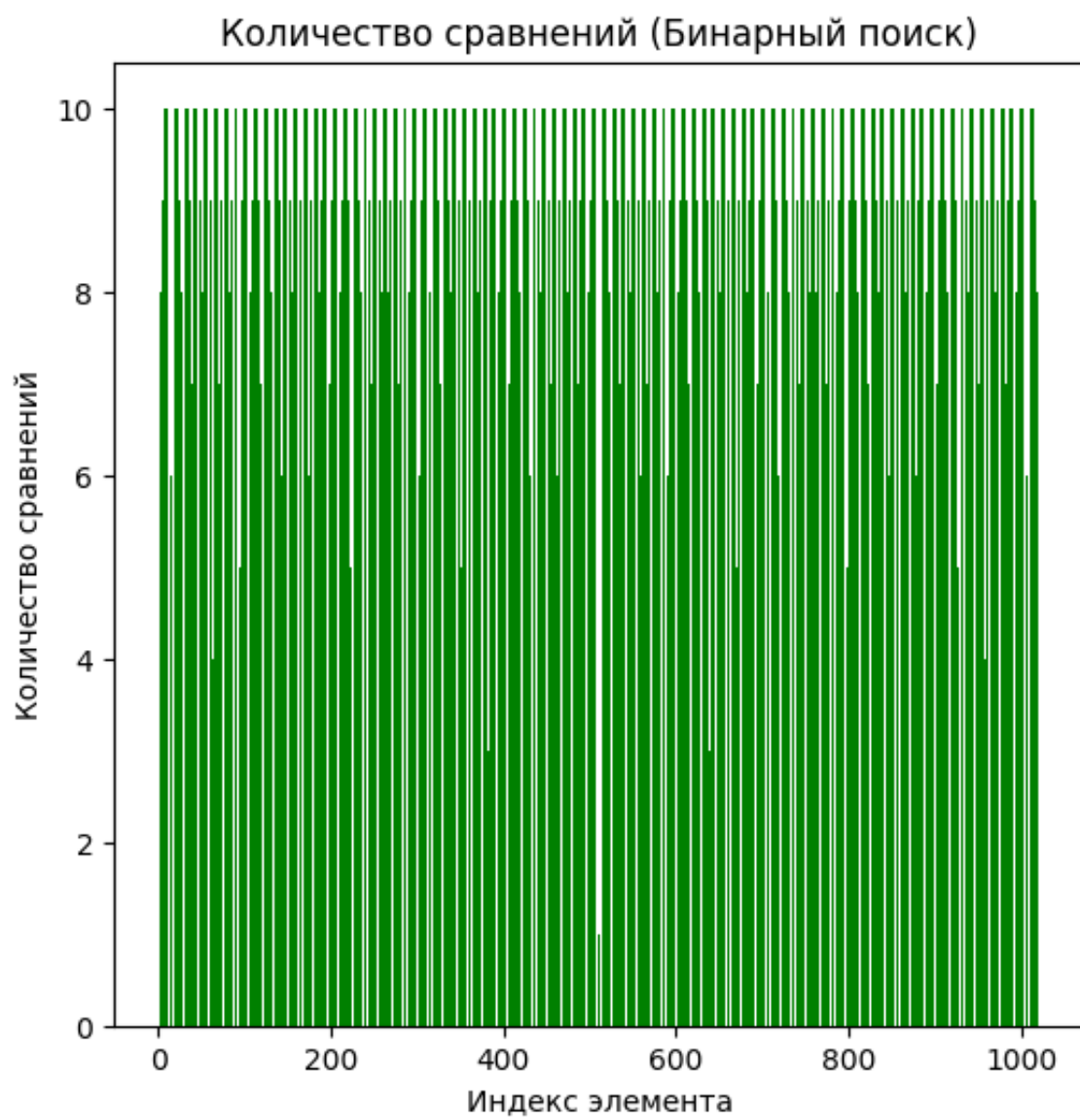
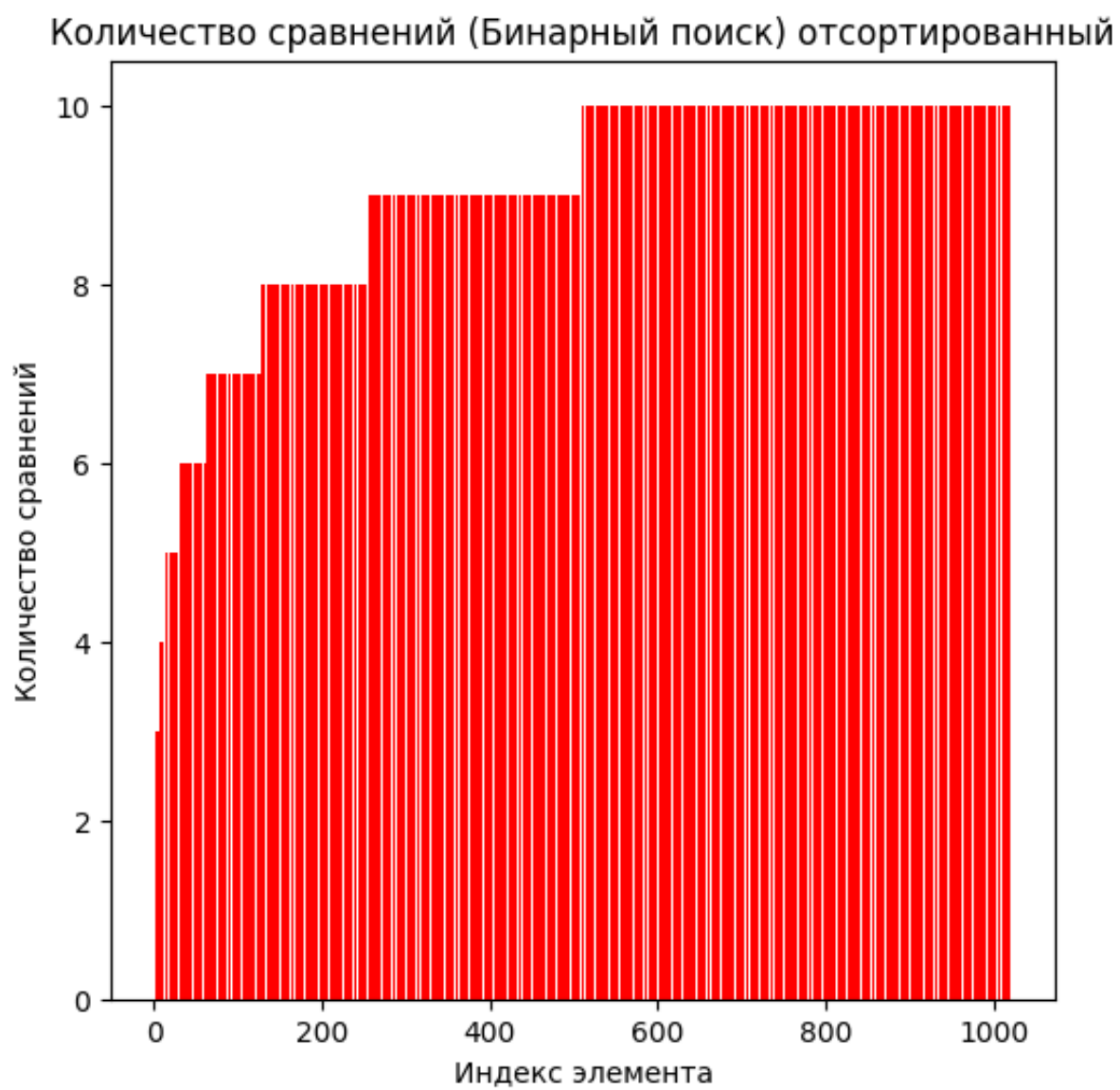


Рисунок 4.2 — Сравнения для каждого элемента в алгоритме бинарного поиска



Вывод

На основании анализа полученных гистограмм для двух алгоритмов поиска в массиве (линейного поиска и бинарного поиска) сделаны следующие выводы:

Линейный поиск: В наилучшем случае, когда искомый элемент находится в начале массива, алгоритм выполняет минимальное количество операций, соответствующее трудоемкости $O(1)$. Как видно из гистограммы, количество операций увеличивается линейно с увеличением индекса элемента. Это полностью подтверждает теоретическую оценку трудоемкости алгоритма в худшем случае, равную $O(N)$, когда необходимо перебрать все элементы массива.

Бинарный поиск: Для бинарного поиска в наилучшем случае (если элемент находится в середине массива) количество операций минимально и соответствует $O(1)$. Гистограмма демонстрирует, что количество сравнений меньше по сравнению с линейным поиском, особенно для элементов с высокими индексами. Трудоемкость алгоритма увеличивается логарифмически по мере увеличения индекса элемента, что подтверждает теоретическую оценку $O(\log N)$. В частности, при размере массива 1023 бинарный поиск завершает работу не более чем за 10 шагов, так как $2^{10} < 1023 < 2^{11}$. При этом, на малых индексах (до 7) линейный поиск иногда оказывается эффективнее, так как для таких случаев требуется меньше операций, чем при первых шагах бинарного поиска.

ЗАКЛЮЧЕНИЕ

В результате работы были подтверждены различия в эффективности алгоритмов поиска в массиве. Бинарный поиск, обладающий сложностью $O(\log N)$, показал более высокую скорость по сравнению с методом полного перебора при работе с большими массивами. Это делает бинарный поиск предпочтительным выбором для работы с отсортированными массивами, где требуются высокие показатели производительности.

Метод полного перебора имеет сложность $O(N)$, что приводит к значительным затратам времени на выполнение поиска, особенно в худшем случае, когда элемент находится в конце массива или отсутствует. Таким образом, для неотсортированных массивов или в ситуациях, где простота реализации важнее эффективности, предпочтительнее использовать метод полного перебора, однако для оптимизации поиска в отсортированных данных бинарный поиск остается более эффективным вариантом.

В ходе выполненной работы были выполнены следующие задачи:

- реализованы алгоритмы поиска значений в словаре;
- проведена оценка трудоемкости этих алгоритмов;
- выполнен сравнительный анализ сложности двух алгоритмов на основе замеров количества сравнений для каждого элемента массива и анализ данных для лучшего и худшего случаев.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Welcome to Python – [Электронный ресурс]. URL: <https://https://www.python.org> (дата обращения: 01.10.2024).
2. Модуль random – [Электронный ресурс]. URL: <https://docs.python.org/3/library/random.html> (дата обращения: 02.10.2024).
3. Библиотека matplotlib – [Электронный ресурс]. URL: <https://matplotlib.org/> (дата обращения: 02.10.2024).